

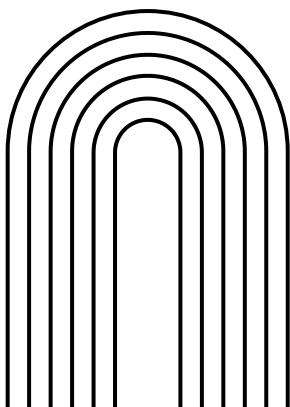
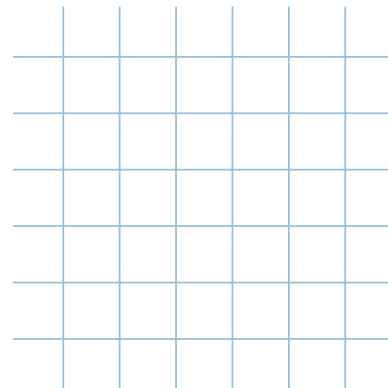
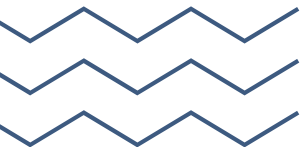
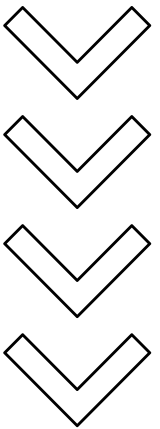
Laura Lucia Jaramillo Yah

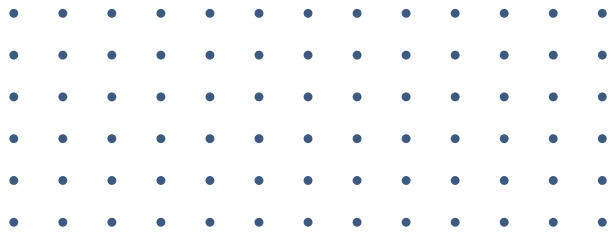
ALGORITMOS DE EXCLUSIÓN MUTUA CON MONTE CARLO

Mtro: Sergio A Noh Puch

Materia: Concurrencia y Paralelismo

Fecha: 09/09/2026





OBJETIVO

Implementar el mismo método de Monte Carlo para calcular π , modificando únicamente el mecanismo utilizado para proteger la variable compartida puntosDentro.

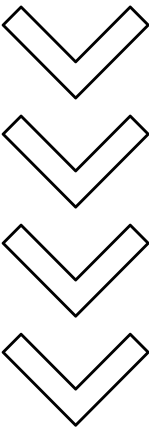
DESCRIPCIÓN

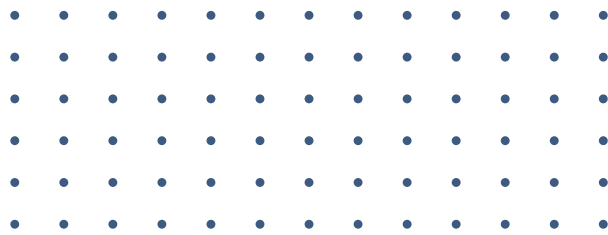
Cada hilo genera una parte de los 100 millones de puntos aleatorios. Cuando un punto cae dentro del círculo, el hilo intenta aumentar el contador compartido mediante CAS. Al terminar todos los hilos, el programa calcula π y muestra cuánto tardó en ejecutarse.

RESULTADOS

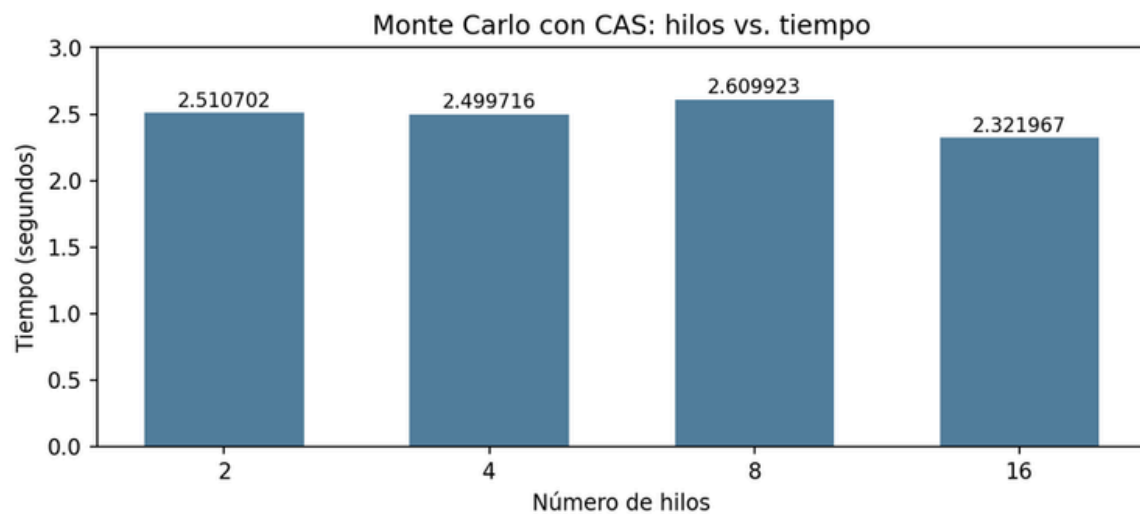
Se mantuvo TOTAL_PUNTOS=1,000,000. Los siguientes datos son los que se dieron después de cada ejecución posterior a la configuración.

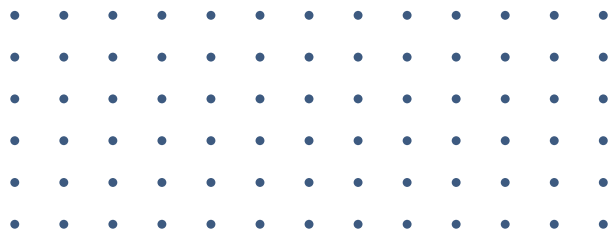
Hilos	Puntos	Pi calculado	Tiempo(s)
2	100,000,000	3.141603	2.510702
4	100,000,000	3.141611	2.499716
8	100,000,000	3.141715	2.609923
16	100,000,000	3.141675	2.321967





```
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ ./montecarlo-clock
Numero de hilos: 2
Total de puntos: 100000000
Puntos dentro del círculo: 78540068
Valor aproximado de Pi: 3.141603
Tiempo de ejecución: 2.510702 segundos
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ gcc -std=c11 montecarlo-clock.c -o montecarlo-clock -pthread -O2
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ ./montecarlo-clock
Numero de hilos: 4
Total de puntos: 100000000
Puntos dentro del círculo: 78540280
Valor aproximado de Pi: 3.141611
Tiempo de ejecución: 2.499716 segundos
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ gcc -std=c11 montecarlo-clock.c -o montecarlo-clock -pthread -O2
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ ./montecarlo-clock
Numero de hilos: 8
Total de puntos: 100000000
Puntos dentro del círculo: 78542882
Valor aproximado de Pi: 3.141715
Tiempo de ejecución: 2.609923 segundos
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ gcc -std=c11 montecarlo-clock.c -o montecarlo-clock -pthread -O2
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ ./montecarlo-clock
Numero de hilos: 16
Total de puntos: 100000000
Puntos dentro del círculo: 78541876
Valor aproximado de Pi: 3.141675
Tiempo de ejecución: 2.321967 segundos
lucy@DESKTOP-TDQ5Q7L:~/CyP_2026/Montecarlo/Montecarlo clock CAS$ _
```





¿CÓMO FUNCIONA CAS?

CAS revisa si el contador todavía tiene el valor que el hilo esperaba. Si es así, lo aumenta en uno. Si otro hilo lo cambió primero, vuelve a intentarlo con el valor actualizado. De esta forma se evitan incrementos perdidos sin usar mutex ni semáforos.

CONCLUSIÓN

En esta serie, 16 hilos terminó más rápido (2.321967 segundos) y 8 hilos tardó más (2.609923 segundos). No siempre se gana tiempo al agregar hilos, porque todos intentan actualizar el mismo contador con CAS. Para comparar bien los resultados, conviene repetir cada prueba varias veces.

